

(a) In this assignment, you have tried to train a FANN using a dataset generated from a bivariate with a bell-shaped response surface. Why does it seem reasonable/possible to use a FANN to model/approximate such a function?

Answer:

It felt reasonable to use a Fully Connected Artificial Neural Network (FANN) for this task because FANNs are universal function approximators. This means that, given enough neurons and proper training, they can approximate any continuous function, including smooth bell-shaped ones like

$$y(x) = e^{-\|x\|^2/16}$$

The structure of the bell-shaped function is smooth, continuous, and has no discontinuities, so it naturally aligns with what neural networks are good at representing. A FANN can flexibly learn curved surfaces and nonlinear patterns without me needing to manually specify the exact mathematical formula. The network simply learns it from data.

In addition, because the function is symmetric and has a smooth gradient, it is not too “irregular” for gradient-based training.

(b) Why is it always dangerous to train without having a validation dataset used for termination of the training (early stopping), and/or for selection between different already trained prediction models/networks alternatives?

Answer:

Working on this assignment showed me clearly why training without a validation set is risky:

- No early stopping; overfitting is almost guaranteed. Without validation monitoring, the model can keep reducing training loss, but performance on unseen data may get worse.
- We can't detect divergence or poor learning. Some learning rates or architectures lead to unstable training. Without validation feedback, we would not know.

- We might select the wrong model. Two networks may look equally good on training data, but only validation data reveals which one generalizes well.

(c) Why is it difficult beforehand to know how simple/complex (for example, number of layers) your FANN model should be in order to be able to approximate the underlying function of interest in sufficient detail?

Answer:

Choosing architecture in advance is difficult because:

- The true underlying function's complexity is unknown.
- Too few neurons, the network underfits and cannot represent the function fully.
- Too many neurons/layers, the network memorizes noise (overfitting).
- The required complexity depends on data difficulty, noise level, smoothness, and even training dynamics.
- Different activation functions behave differently (e.g., ReLU sometimes struggles with smooth curves).

(d) Why is it difficult to know how well a particular FANN model, which has a particular architecture with particular activation functions, can learn successfully from a given dataset? Hint: Overfitting and underfitting in relation to size of dataset and model flexibility.

Answer:

This difficulty comes from the interplay among:

1. Model size (capacity): A small model cannot learn a complex function, underfitting. A large model may memorize noise, overfitting.
2. Dataset size: 1000 training examples vs 100 examples creates completely different learning conditions. Small datasets create high variance in performance.
3. Noise: If noise is high, the network may overfit unless regularized.

4. Training dynamics: Learning rate, initialization, and randomness affect performance unpredictably.
5. Mismatch between architecture and function: For example, smooth bell curves sometimes train better with tanh than ReLU.

This makes it difficult to fully predict learning success ahead of time, experimentation is required.

(e) What did you find frustrating and/or difficult, and/or unnecessary, and why?

Answer:

Some challenges I encountered:

1. The need to regenerate datasets repeatedly.
2. The fine-tuning of hyperparameters. Learning rates and the number of epochs made a big difference, and I had to experiment a lot.
3. Writing long code for 3D surface plots. It felt repetitive, and sometimes unnecessary, especially when debugging.

However, all these difficulties did help reinforce my understanding.

(f) What would you suggest as an improvement of this assignment for next years students, and why?

Answer:

I'd suggest adding a small Q&A section describing common mistakes. This could reduce the unnecessary time spent debugging.

(g) Did you spend more than 20 hours to complete this exercise? Why?

Answer:

Yes, I spent more than 20 hours on this assignment.

Main reasons:

- Learning and debugging PyTorch code took time.
- Tasks D and E required multiple model runs, which is time-consuming.
- Understanding model behavior (overfitting, early stopping, architecture choice) required trial-and-error.
- Creating surface plots added extra work.